SECURITY AND TRUST

# Signing Keys by Proxy

**Buy applications were built for one document format at a time:** SAP spreadsheets, Adobe Experience Cloud and AEM pages, browser-managed data. Extending to AI and robots reaches households, warehouses, agents and peer mandates. **Rules have to run as memory-safe Rust code with securely random keys, not fail at a document-type rule.** Security counts the most where humans matter the most: in our households.

---

Every signature answers two questions: *who issued the key*, and *who granted the signer the right to use it*. When a consultancy, a plugin or a platform signs on your behalf with a certificate authority (CA) you never chose, you inherit that CA by proxy: its trust, its revocation service, its jurisdiction, and its failures. October 2026 showed what that costs when the CA is distrusted. This article sets out when you use a CA by proxy, what locks out when its chain breaks, how ECC/SAP signing fits, what each market requires, and how to build a Globalization MVP security first, with a cutover checklist to get there.

## What are signing keys, and where are they used?

A signing key is a business's digital seal. It comes in two halves. The **private key** stays with whoever signs, like the stamp locked in a notary's drawer. The **public key** is shared with everyone, so anyone can check that a stamp is genuine without asking the signer. A valid signature proves two things: **who** signed, and that **nothing changed** after they signed. Change one character of a signed contract, firmware file or code release and the check fails.

You already rely on signing keys every day, usually without seeing them:

| WHERE | WHAT IS SIGNED, BY WHOM | WHAT IT PROVES | IF THE KEY FAILS |
|---|---|---|---|
| **Code on GitHub** | Commits and pushes, with a developer's RSA or Ed25519 key | Who made the change, and that it wasn't altered | Anyone with the key can push as that person: revoke it and issue a new one |
| **Firmware and SBOMs** | The firmware image and its parts list (SBOM), by the manufacturer | The device runs the real update, and the parts list matches what shipped; the EU Cyber Resilience Act expects this | Devices reject updates, or accept a fake one |
| **E-signatures on legal documents** | Contracts, SOWs and filings, with a certificate from a certificate authority (CA) | This person agreed to this exact document, at this time | Signed documents show as invalid if the CA is distrusted |
| **QA attestations in Agile** | Review approvals, test runs and release builds, by reviewers and the CI pipeline | The build was reviewed and tested, and no gate was skipped | No proof a release passed QA: an audit gap |

**The business question is never only "is it signed?"** It is also: *whose key, issued by whom, and who granted them the right to sign for us?* A developer's key proves who pushed, but not whether they were allowed to push to that repository. A consultant's certificate proves who signed, but not whether they could sign for your company. That gap between **issuer** and **grant** is what this article is about, and it grows when the key belongs to someone else: a vendor, a plugin, or a consultancy signing **by proxy**.

#### Why this matters now: keys have to move on events

Historically, signatures lived inside SLA-bound enterprise applications with a heavy price tag, such as SAP and AEM, and governance of those keys was assigned to a consulting team. The keys sat inside the application, and changing who could sign meant a change request, a consulting ticket and a release window. That worked when signing happened a few times a quarter.

It also fitted what those systems were built for. SAP grew from accounting: accumulated ledger rows, in effect spreadsheets at enterprise scale, posted in batches by trusted staff on an internal network. Records like that never needed memory safety, because nothing untrusted reached the code at run time. **Attestation was a timestamp**, the posting date and the change log, not a signature anyone outside could verify. **Data subject request (DSR) evidence was held in pay-to-use data repositories**: vendor add-ons and archives billed by volume or by query, so proving a deletion meant paying to read your own records.

| | THEN: SLA APPLICATION | NOW: EVENT-DRIVEN AI |
|---|---|---|
| **Records** | Ledger rows posted in batches by trusted staff | Events from shoppers, partners and agents, arriving all the time |
| **Runtime exposure** | Internal network; memory safety not a requirement | Internet-facing edge, where memory-safe code (Rust, Wasm) is the baseline |
| **Attestation** | A timestamp and a change log | A signature anyone can verify offline, against a key you publish |
| **DSR evidence** | Held in pay-to-use repositories | An append-only ledger you own, with a receipt from every system that acted |
| **Key changes** | Change request, consulting ticket, release window | Issued, rotated or revoked on the event |

AI work signs on every event: each pull request, each consent change, each new market. Three capabilities stay blocked until the organization can **exchange keys on the event**, issuing, rotating or revoking a key or grant at the moment something happens:

| AI CAPABILITY | WHAT IT HAS TO SIGN ON EACH EVENT | BLOCKED WHEN KEYS LIVE IN AN SLA APPLICATION |
|---|---|---|
| **AI code review** | A review attestation per pull request, with a short-lived key the CI pipeline can verify | Keys are held in vendor or consultancy tooling, with no way to issue one per review |
| **Data governance by mandate** | A scoped, expiring mandate on every agent action, revoked the moment consent changes | Grants change through tickets and release windows, not on the consent event |
| **AI globalization weighting** | Each market's weighting or adapter, published as a signed record when that market's rules or data change | Weights and configuration ship on the application's release cycle, so a market can't be signed for when its rule changes |

**The risks of leaving keys there.** When signing and its governance sit with an external team inside an SLA application, the organization carries risks it doesn't control:

| RISK | WHAT HAPPENS | WITH KEY EXCHANGE ON EVENTS YOU OWN |
|---|---|---|
| **Single point of failure: one developer** | The one person who holds a key, or knows how to rotate it, is away or leaves. Releases and revocations stall | Keys live in the organization's key store, not with a person; rotation needs two approvers and follows a written runbook |
| **External supply chain access** | A per-seat plugin or application, run by an outside team, holds standing credentials into your systems. Their compromise becomes yours | Each vendor gets a scoped, expiring grant and its own certificate, revocable alone, and every access is a signed record |
| **Knowledge and training data lost at the end** | When the subscription or the external team's contract ends, the knowledge base, prompts, labelled data and adapters stay on their platform | The corpus is published as signed records you hold; adapters and training sets sit in your own registry; exit terms require export |
| **Time zone gap** | An incident starts while the external team is offline, and nobody on your side can rotate or revoke | Revocation is automated on the event, and your own on-call staff hold pre-approved emergency mandates |
| **Crisis mitigation on breach** | During a breach, nobody can say who holds which key, revoke it quickly, or show regulators what happened when. GDPR allows 72 hours to notify | One switch per key and per grant; revocation reaches every system on the event; the ledger's timestamps are the evidence for the notification |
| **Plugins without memory safety, with hard-coded upserts** | A plugin or application writes straight into the database with hard-coded upserts, outside the event path: no signature, no consent check, no ledger entry. A memory bug in it can expose keys held in the same process | Plugins hold no signing keys and no direct write access; every write passes the gate as a signed event; the gate runs memory-safe code (Rust, Wasm) |
| **Unsigned device commands and soft firmware: households** | Home appliances accept commands from anyone who reaches their login page, as the supermarket controllers did in October 2026, and stream usage data that shows when the home is empty | An encrypted streaming gate the household controls, a certificate per appliance with household-held keys, signed commands within firmware limits, and hardened, signed firmware |

**Distributed keys and rolling data gating.** The answer to these risks is to spread keys across the systems that act, each with its own scoped grant, and to have AI check every write against consent, market rules and mandates as it arrives: a gate that rolls with the data instead of a batch review after it. That model only holds if nothing writes around the gate. A plugin that hard-codes upserts into the data store, or runs in a runtime without memory safety, is a side door: its writes carry no signature, and a single memory-corruption bug can hand an attacker the keys and the write path together. October 2026 made the point twice: php2shell gave unauthenticated code execution in the PHP runtime under every WordPress, Drupal, Magento, Joomla and Laravel host, whatever the plugin code, and an AI gateway package was compromised for the second time this year, again stealing signing keys and AI provider keys from the environment it ran in.

Key exchange on events is what moves signing out of the SLA application and into infrastructure the organization owns. The rest of this article shows what that takes: whose keys, which grants, and where each market's rules apply.

# Does the client use eMudhra by proxy?

When a consultancy signs with an eMudhra certificate, it depends on whose certificate signs and who runs the system.

THREE PARTIES

**Client** — RELYING PARTY
Hires the consultancy
Opens signed PDFs; may have its own CA

hires →
← signed PDF

**Consultancy** — E.G. BIG FOUR OR IT SERVICES
Signs its own documents
May run signing or systems for the client

certificate from →

**eMudhra** — CERTIFICATE AUTHORITY
Issues signing certificates
Runs revocation checks (OCSP, CRL)

FOUR SITUATIONS, FOUR ANSWERS

### Signs its own — NO

Consultancy signs → client validates

An SOW, invoice or sign-off signed with the consultancy's eMudhra certificate.

The client only checks the chain and revocation. No account, key or contract with eMudhra.

### Signs via their setup — YES, INDIRECTLY

Client signer → consultancy → eMudhra

Client people sign through the consultancy's Acrobat Sign account or eSign portal.

Signer identity data goes to eMudhra: effectively a subprocessor the client never chose.

### Runs client systems — YES, DIRECTLY

eMudhra cert in the client's AEM or Sign

The consultancy installs eMudhra certificates or enables it as the provider in client systems.

It is now the client's configuration; the consultancy only made the choice.

### Client has its own CA — NO

Client signs with its own CA

The client only receives eMudhra-signed documents.

It needs to trust eMudhra's root, usually through Adobe's trust list (AATL).

EVEN AS A RELYING PARTY ONLY, THE CLIENT DEPENDS ON EMUDHRA FOR

### Revocation checks

Validating a signature calls eMudhra's OCSP responder or CRL. Long-term validation helps only if it was embedded at signing.

### Trust in the root

Not hypothetical: 17–20 Oct 2026, Chrome, Mozilla, Apple and Microsoft blocked eight eMudhra intermediates whose private keys were recoverable. Chrome removes emSign ECC Root CA - G3 from 27 Oct.

### Legal weight

Valid under Indian law. Whether it counts as qualified in the EU, or elsewhere, depends on the certificate type and jurisdiction.

AI / TRUST / BOUNDARY LOCKOUT: WHAT STOPS WHEN A CHAIN IS DISTRUSTED

### AI lockout — AGENTS

Agent → TLS handshake → blocked chain → no call

Agents, AI gateways and MCP tools calling an API whose certificate chains through a blocked intermediate fail the handshake: the tool call never happens.

Mandates signed with keys from an exposed environment must be rejected, and code-signed packages and adapters fail verification.

### Trust lockout — RUNTIMES

Browser, OS, Node, Composer, certifi reject the chain

A distrusted chain is not a warning: browsers and runtimes refuse the connection, so the site or API is unreachable.

Signed documents on that chain show as invalid wherever they are opened.

### Boundary lockout — CROSSINGS

Edge ↔ origin, webhooks, feed syncs, partner APIs

Every boundary crossing is a TLS handshake. A distrusted chain on either side stops the crossing: feeds stall, webhooks fail.

Consent, opt-out and deletion events stop propagating: a DSR lockout. Queue them and replay once the chain is reissued.

Recover faster: know which certificates chain through which CA, automate reissue (ACME) with a second CA ready, and sign records with keys you own and can verify offline.

WHEN IT IS BY PROXY, THESE BECOME CONCRETE

### Vendor and subprocessor lists

eMudhra belongs in the client's third-party register and data processing agreement, through the consultancy's contract.

### Personal data

Signer identity checks, including Aadhaar eSign, process personal data in India: residency and transfer questions for EU and UK signers.

### Exit

If the consultancy leaves, the signing setup and certificate relationship may go with it.

CONTRACT CLAUSES TO ASK FOR

### Disclose every provider

Every trust service provider and CA used for the client's signers or systems.

### Certificates issued to the client

Certificates installed in client systems name the client, not the consultancy.

### Embed LTV

Revocation info and timestamps embedded so documents stay verifiable after the relationship ends.

CERTIFICATES YOU CONTROL: NO THIRD-PARTY CHAIN TO INHERIT

### crm-sync certificate link — OWN KEY

crm-sync.dev/license/verify?cert=…

Licence grants, upload attestations and deletion records are signed with crm-sync's Ed25519 key, published at /.well-known/jwks.json.

Anyone verifies offline, with no account and no CA chain that can be distrusted. Short links at /v/<hash>; print QR at /license/qr/<hash>.svg.

### Xano self-host certificate — YOUR CA

Your domain, your TLS certificate, your trust store

Self-hosted Xano runs on infrastructure you operate: you choose the CA for its TLS certificate and keep the private key in your own HSM or key vault, in country.

You also own the trust store: confirm the Node and Composer bundles reject the October 2026 eMudhra chains.

ECC / SAP SIGNING: THE SAME SCRUTINY, PLUS TWO ECC-SPECIFIC RISKS

| What gets signed | Mechanism in ECC | Who usually sets it up | Proxy risk |
|---|---|---|---|
| **PDF forms**<br>Invoices, POs, delivery notes | Interactive Forms by Adobe through Adobe Document Services (ADS), with a .pfx credential | Consultants, during the build | Directly: the CA behind that credential is now the client's |
| **Approvals and records**<br>QM, batch records, change master, 21 CFR Part 11 | SAP digital signature through SSF, keys in PSE files managed in STRUST | Basis team or consultants | Directly, if they imported keys or certificates from their CA |
| **Outbound HTTPS**<br>Shopify, Google, banks, tax portals | ICM web server, SSL client PSEs in STRUST | Basis team or consultants | Trust store: ECC may still trust a chain everything around it now refuses |
| **India statutory filings**<br>GST, e-invoice, company filings | Digital Signature Certificates (DSCs) on USB tokens; eMudhra is a large issuer | Often the consultancy or an outside CA firm | Grant problem: the certificate names a consultant, not the client |
| **Spreadsheets**<br>Migration templates, LSMW, Migration Cockpit, reports | Usually unsigned; sometimes Office signatures with a personal certificate | Consultants | No signature at all, so no provenance |

### STRUST doesn't update itself

Browsers, Node, Composer and operating systems shipped the eMudhra distrust in October. An ABAP trust store changes only when someone edits it.

Check every PSE in STRUST, plus ADS and Java stack keystores.

### A certificate is not authority

A CA proves who holds the certificate, not what they may sign. A consultant's DSC on a client filing needs a written grant: who, what, how long, revoked on exit.

### The S/4HANA move carries trust

A system conversion (SUM with DMO) migrates the PSEs too. Clean out stale or proxy certificates first; ECC mainstream maintenance ends in 2027.

### Unsigned spreadsheets

Migration data handed over by consultants has no issuer. Hash each handoff file and record it, signed with a key you own: the same ledger as firmware.

### Checklist for the consulting engagement

- Export every PSE and certificate in STRUST, with its issuer chain and who imported it
- Name the ADS signing credential: issued to whom, by which CA
- List every DSC used for statutory filings, who holds the token, and the grant behind it
- Require certificates in client systems to name the client
- Hash and log every spreadsheet and PDF handed over

RULES MANAGEMENT: IN-COUNTRY DATA AND TRANSFER, WITH CLOUDFLARE AND XANO

| Country | Law | Data in country | Cross-border transfer | Cloudflare | Xano |
|---|---|---|---|---|---|
| China | PIPL, Data Security Law, Cybersecurity Law | Required for critical infrastructure operators and above volume thresholds; important data stays | Security assessment, standard contract or certification, unless an exemption applies | China Network via JD Cloud (Enterprise), needs an ICP filing; Geo Key Manager picks key regions | No mainland region: self-host on a mainland cloud, with its own TLS certificate from a CA you choose |
| Japan | APPI | Not required by law | Consent naming the country, or a recipient with equivalent safeguards; EU adequacy | Regional Services and Geo Key Manager keep TLS processing and keys in Japan | A Japan-region instance where offered, or self-host in Japan |
| Korea | PIPA (2023 amendment) | No general rule; sector rules for finance and public-sector cloud | Consent before transfer, or a certified or adequate recipient; disclose country and recipient | Regional Services and Geo Key Manager; check the Korea region | A Seoul-region instance, or self-host in Korea |
| Vietnam | PDP Law (from 2026), Decree 53 (Cyber Law) | Some services must store in Vietnam; foreign firms when ordered by the regulator | Transfer impact assessment filed with the Ministry of Public Security | No local residency product; keep personal data out of edge logs and caches | Self-host in Vietnam where localisation applies; otherwise a nearby region with the filing |
| Singapore | PDPA | Not required by law | Comparable protection: contract, binding corporate rules or certification | Regional Services and Geo Key Manager; check the region list | A Singapore-region instance where offered |
| Taiwan | PDPA (2023 amendment) | Not general; sector rules | Allowed unless the regulator restricts a destination, e.g. mainland China for some sectors | Standard network; keep keys out of restricted regions | Nearest region or self-host; never route through a restricted destination |
| EU · Germany | GDPR, BDSG, TDDDG (cookies) | Not required by law; often a contract term for public sector and health | Adequacy (incl. EU-US DPF, Japan, Korea, UK) or SCCs with a transfer assessment | Data Localization Suite: Regional Services, Customer Metadata Boundary, Geo Key Manager (EU) | An EU-region instance, or self-host in Germany |
| UK | UK GDPR, DPA 2018, Data (Use and Access) Act 2025 | Not required by law | UK adequacy regulations, IDTA or Addendum; UK-US data bridge | Regional Services; check the UK region | A UK or EU-region instance, or self-host in the UK |

A signature you only validate is a dependency. A signing setup your people use, or a certificate in your systems, is a vendor.

Patterns, not a claim about any firm. eMudhra incident per The Hacker News and Help Net Security, October 2026. Country rules summarised; check Cloudflare and Xano region lists. Not legal advice.

Yoonsun Lee · YSL150 LLC

# 1. When you use a CA by proxy

Three parties are involved: the **client** (you), the **consultancy** or vendor, and the **CA** that issued the consultancy's certificate. Whether you use that CA depends on whose certificate signs and who runs the system.

| SITUATION | DO YOU USE THE CA? | WHY |
|---|---|---|
| The consultancy signs its own documents (SOW, invoice, sign-off) | **No** | You are a relying party: you validate the chain and revocation. No account, key or contract with the CA |
| Your people sign through the consultancy's setup (its Acrobat Sign account or eSign portal) | **Yes, indirectly** | Your signers' identity data goes to the CA: effectively a subprocessor you never chose |
| The consultancy builds or runs your systems and installs the CA's certificates in your AEM, Acrobat Sign or SAP | **Yes, directly** | It is now your configuration; the consultancy only made the choice |
| You sign with your own CA and only receive the consultancy's signed documents | **No** | You need to trust the CA's root, usually through Adobe's trust list (AATL) or the OS store |

Even as a relying party only, you depend on the CA for **revocation checks** (its OCSP responder or CRL; long-term validation helps only if it was embedded at signing), **trust in its root**, and the **legal weight** of the signature in your jurisdiction.

These are patterns, not claims about any firm. A contract, or the certificate chain of a signed PDF, shows which provider a given consultancy uses.

## 2. Why it is not hypothetical: October 2026

A key ceremony at eMudhra on 30 April 2025 generated eight intermediate CA keys with too little randomness. Between 17 and 20 October 2026 a researcher recovered all eight private keys from the public certificates, and Chrome, Mozilla, Apple and Microsoft blocked them: six TLS intermediates and two

code-signing intermediates. Chrome removes the emSign ECC Root CA - G3 from devices from 27 October. Go, NSS, Ubuntu, Debian and Composer shipped trust-store fixes on 20 October, and the Node.js releases of 21 October also reject the chains. India's Digital Signature Certificates run under India's own CA licensing rather than the browser root programs and, as reported, were not directly affected.

The keys were not stolen; they were computable. An audited process with certified hardware still produced weak keys, so the lesson is about provenance: know which of your certificates chain through which CA, and sign your own records with keys you own and can retire. Background: Hydrogen or bespoke stack, Org Trust Layer, and reporting by The Hacker News and Help Net Security.

#### The same month, on the shop floor: refrigeration controllers in France and the UK

The eMudhra keys were weak; in October's other incident there were no keys at all. On the night of 11 October 2026, attackers reached the refrigeration controllers of hundreds of French supermarkets, including Intermarché, Netto, Coopérative U, Carrefour, Monoprix and E.Leclerc stores. The Danfoss controllers ran web servers open to the internet, and many accepted the username `admin` with the current date as the password. The attackers set cases to defrost with food inside: some reached about 50 °C within six hours, some equipment melted or caught fire, and the change went unnoticed because it raised no alarm and no staff were present. No data was stolen; the losses ran to tens of millions of euros. Danfoss had shipped a fix for its current AK-SM800A series in August 2025, but older units were end-of-life. The Prime Minister asked ANSSI to investigate, and its director said the controllers urgently needed disconnecting from the internet.

On 13 October, UK Cyber Defence found more than 800 UK refrigeration and building controllers with their login pages on the open internet. By early

November, UK victims were confirmed: a West London wholesaler, Langridge Organic, found six of its ten cold-room controllers, running Copeland's Dixell XWEB software, set to defrost with a 150 °F maximum and their temperature alerts switched off. Manual checks saved its stock. The NCSC said it was aware of a small number of affected UK organisations. On 2 November, similar attacks hit supermarket refrigeration in Belgium, Luxembourg, the Netherlands, Sweden, Estonia, Ireland and Portugal. No one has been publicly identified as responsible.

Both incidents are the same signing-keys problem. A controller that accepts any command from anyone who reaches its login page has no **issuer** (no device or operator certificate) and no **grant** (nothing says who may change a setpoint, by how much, or when). The attacker also switched off the alarm through the same channel, so the record of what changed was lost with it. Sources: Wikipedia, Public Sénat, and The Grocer on UK exposure and UK victims.

#### Households: where humans matter the most

Security counts the most where humans matter the most: in our households. The October attacks hit shops, but the same class of device sits in homes: fridges and freezers, heat pumps, boilers, thermostats and EV chargers, all on the household network and most talking to a manufacturer's cloud. A home has no security team and no morning temperature check. If a supermarket controller can be set to defrost from the internet, a home appliance with an open login page or a default password can be too, and its data stream says far more about the household: when the fridge door opens, when the heating comes on, when nobody is home.

**Why households wire them up.** The benefits are real, and they are why the answer is to secure the stream, not to unplug:

| USE | BENEFIT TO THE HOUSEHOLD |
|---|---|
| **Heating, heat pumps and thermostats** | Lower bills: heating runs on schedules and on cheaper time-of-use tariffs, and only in rooms in use |
| **Fridges and freezers** | Less food waste: a warning when the door is left open or the temperature drifts |
| **Cameras, doorbells and baby monitors** | Watching over children, pets and deliveries while away |
| **Sensors for older relatives** | Living independently for longer, with an alert to family if something changes |
| **EV chargers and home batteries** | Charging when power is cheapest and cleanest |
| **The TV and phone together** | One screen and one remote for the whole home |

**The TV and the phone have become the home's micro-network.** Many households now run the home from a phone on the sofa and a smart TV on the wall: casting to the screen, using the phone as the remote, and letting the TV act as the hub for lights, cameras, speakers and appliances. Newer devices join a local mesh (Thread) and a common smart-home standard (Matter), so the phone, the TV and each device talk inside the home, without a round trip to every maker's cloud. That is good for speed, for working when the internet is down, and for privacy.

It also makes the TV and the phone the **gate**. Whatever reaches them can reach every device paired to them, and TVs often run for years without updates while streaming viewing data to their makers. Matter shows the right pattern: every certified device carries its own **device attestation certificate** from its manufacturer, and pairing gives it a fresh **operational certificate** issued by the household's own network, so the household, not the maker, holds the keys that let a device join or leave. The household's gate should work the same way for everything else: local first, encrypted, and under keys the household holds.

Cameras show what is at stake. In late November 2025, Korean police arrested four people over the hacking of about **120,000 internet-connected cameras** in

homes and businesses, including private homes, a Pilates studio and a gynecology clinic. Two of them accounted for most of the hacks and made sexually exploitative videos from the footage, which were sold on an overseas site; police also arrested buyers and moved to close the site. The way in was simple: the cameras had weak passwords, such as repeated characters or sequential numbers and letters. Many had been installed to watch over children and pets. A leaked video can't be recalled the way a spoiled freezer can be restocked, which is why the stream itself has to be gated. Sources: BleepingComputer, Kaspersky, The Star.

Households need two things before anything else: **an encrypted data streaming gate**, and **hardened firmware**.

| REQUIREMENT | WHAT IT MEANS AT HOME | WHAT IT STOPS |
|---|---|---|
| **No open login page, no default password** | Each appliance ships with a unique credential and no management page on the internet | The October method: find the page, type `admin` and the date |
| **An encrypted data streaming gate** | Telemetry and camera video leave the home only encrypted, through one gate the household controls, and only for the purposes the household agreed to | Usage data revealing when the home is empty; silent streaming to parties the household never chose |
| **Keys held by the household** | Every appliance has its own certificate, and the household holds the keys that pair it to apps and services, so access can be revoked one device or one service at a time | A manufacturer, installer or app keeping standing access after the household has moved on |
| **Signed commands within safe limits** | Remote commands from an app or cloud are signed, checked against what the household granted, and kept within limits set in firmware | A fridge told to run a 50 °C defrost, or a heating system turned off in winter |
| **Hardened firmware** | Updates are signed and verified before install, the device boots only signed code, memory-safe components handle network input, and an SBOM lists what is inside | A forged update, a memory-corruption bug becoming remote control, or an unpatched unit nobody knew was vulnerable |
| **A stated support period** | The maker says how long security updates will come, and the device warns before they stop | Appliances that keep running years after their last patch, as the end-of-life French controllers did |
| **The TV and phone as the gate** | The hub (TV, phone or home hub) gets security updates for as long as it runs the home, appliances sit on their own network segment, and pairing uses the household's own certificates (as Matter does) | An old, unpatched TV becoming the way into every device paired to it |
| **Local alarms and local control** | Alerts sound on the device and work without the cloud; the cloud can't switch them off | An attacker silencing the warning while changing the setting |

The rules are arriving. The UK's product security regime has banned universal default passwords on connectable consumer products since April 2024 and requires a stated support period. In the EU, the Radio Equipment Directive's cybersecurity requirements have applied to wireless devices since August 2025,

and the Cyber Resilience Act adds vulnerability reporting from September 2026 and full security obligations, including secure updates and an SBOM, from December 2027.

The same controls protect lights-out sites, where cold rooms, chillers and server cooling run with nobody on the floor; see Entitlement Security.

## 3. AI security: trust and boundary lockout

A distrusted chain is not a warning. Browsers and runtimes refuse the connection, and everything that depended on it stops.

| LOCKOUT | WHAT STOPS | SIGNAL |
|---|---|---|
| **AI lockout** | Agents, AI gateways and MCP tools calling an API whose certificate chains through a blocked intermediate fail the TLS handshake, so the tool call never happens. Mandates signed with keys from an exposed environment must be rejected; code-signed packages and adapters fail verification | Denied tool calls, handshake errors in agent logs |
| **Trust lockout** | Browsers, operating systems, Node, Composer and Python's `certifi` refuse the chain: the site or API is unreachable. Signed documents on the chain show as invalid wherever they are opened | Unreachable origins, invalid signature banners |
| **Boundary lockout** | Every crossing (edge to origin, webhooks, feed syncs, partner APIs) is a TLS handshake. Feeds stall and webhooks fail. Consent, opt-out and deletion events stop propagating: a **DSR lockout** | Stalled feeds, growing webhook retries, unacknowledged deletions |

**Recover faster:** keep an inventory of which certificates chain through which CA, automate reissue (ACME) with a second CA ready, queue DSR events so they can be replayed with their timestamps, and sign records with keys you own so they still verify when a public CA does not.

# 4. Tools and permissions: a certificate is not a grant

A CA proves who holds a certificate. It does not prove what they may sign or do. That needs a **grant** from the organization: who, what, for how long, revoked on exit. For AI agents the grant is a **mandate**: scoped to tools and markets, expiring, and checked on every action.

### Certificates you control

| | CRM-SYNC CERTIFICATE LINK | XANO SELF-HOST CERTIFICATE |
|---|---|---|
| What it is | Licence grants, upload attestations and deletion records signed with crm-sync's Ed25519 key, published at `/.well-known/jwks.json` | Self-hosted Xano on infrastructure you operate, with a TLS certificate from a CA you choose |
| How it is checked | `crm-sync.dev/license/verify` : anyone verifies offline, with no account and no CA chain that can be distrusted; short links at `/v/<hash>`, print QR at `/license/qr/<hash>.svg` | Your trust store: confirm the Node and Composer bundles reject the October 2026 eMudhra chains |
| Key custody | The platform's signing key; the public half is published | Your own HSM or key vault, in country |

Assets that prove provenance (PDFs and firmware, images, 3D files, docs) are vaulted the same way: each file under its own key, every upload certified, every download granted and recorded in a hash-chained ledger. See the Protected Firmware Asset Management and Download Service and Firmware asset publishing.

### ECC/SAP signing: the same scrutiny, plus two ECC-specific risks

| WHAT GETS SIGNED | MECHANISM IN ECC | WHO USUALLY SETS IT UP | PROXY RISK |
|---|---|---|---|
| PDF forms (invoices, POs, delivery notes) | Interactive Forms by Adobe through Adobe Document Services (ADS), with a `.pfx` credential | Consultants, during the build | Directly: the CA behind that credential is now yours |
| Approvals and records (QM, batch records, change master, 21 CFR Part 11) | SAP digital signature through SSF, keys in PSE files managed in `STRUST` | Basis team or consultants | Directly, if they imported keys or certificates from their CA |
| Outbound HTTPS (Shopify, Google, banks, tax portals) | ICM web server, SSL client PSEs in `STRUST` | Basis team or consultants | Trust store: ECC may still trust a chain everything around it now refuses |
| India statutory filings (GST, e-invoice, company filings) | Digital Signature Certificates on USB tokens | Often the consultancy or an outside CA firm | Grant problem: the certificate names a consultant, not the client |
| Spreadsheets (migration templates, LSMW, Migration Cockpit, reports) | Usually unsigned; sometimes Office signatures with a personal certificate | Consultants | No signature at all, so no provenance |

- `STRUST` **doesn't update itself.** Browsers and runtimes shipped the distrust in October; an ABAP trust store changes only when someone edits it. Check every PSE in `STRUST`, plus ADS and Java stack keystores.
- **The S/4HANA move carries trust.** A system conversion (SUM with DMO) migrates the PSEs too. Clean out stale or proxy certificates first; ECC mainstream maintenance ends in 2027.
- **Unsigned spreadsheets are the weakest link.** Hash each handoff file and record it, signed with a key you own.

**Stores: POS registers in sync with warehouse and RMA systems**

A point-of-sale register is a device that moves money and stock. When registers, the warehouse management system (WMS) and the returns system (RMA, return merchandise authorization) sync by batch file or shared credentials, nobody can say which register recorded a return, who authorised the refund, or whether the item ever reached the warehouse. Keeping them in sync safely means every step is a **signed event** from a known device, under a grant, and each system acts only on events it can verify:

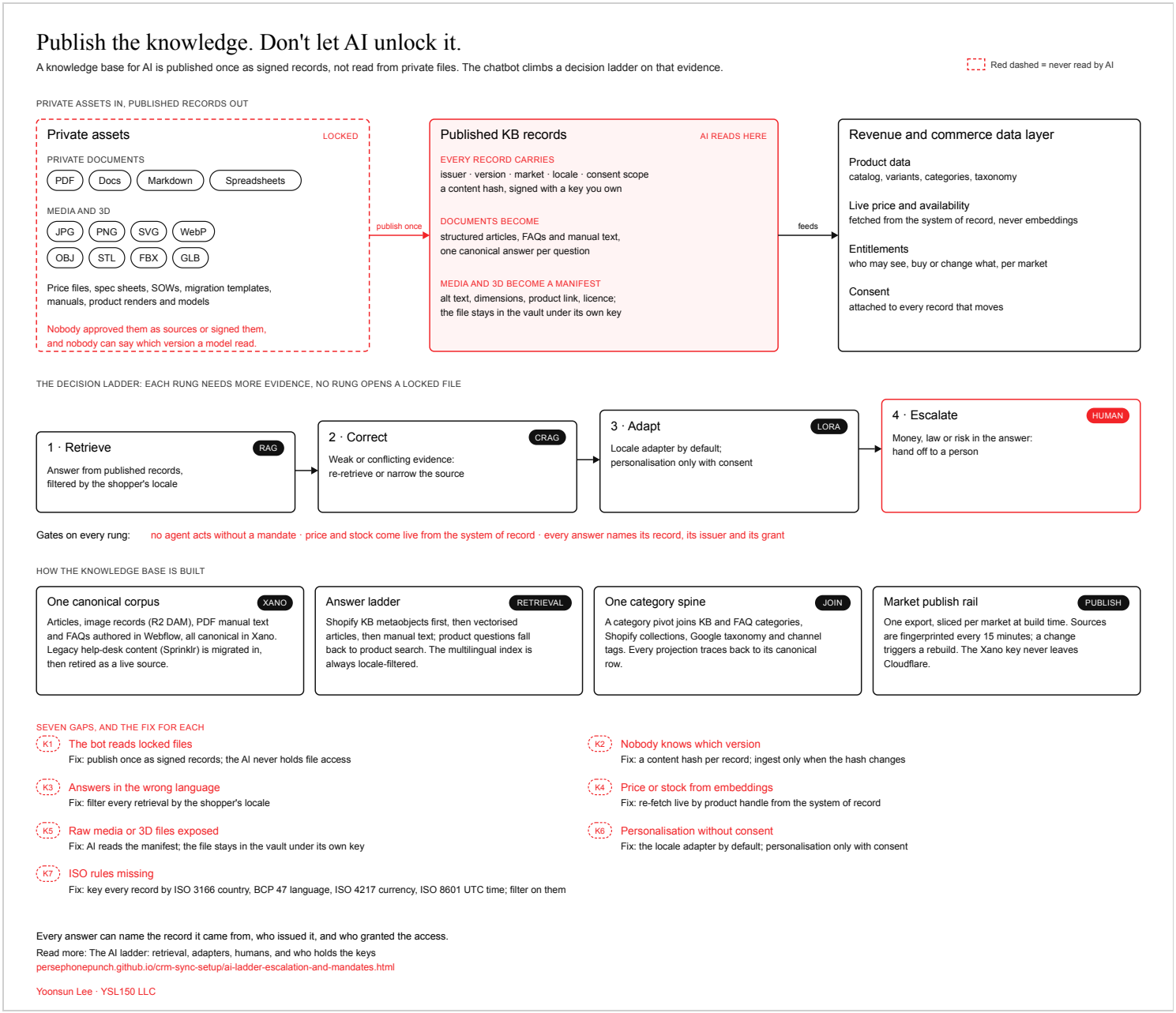
| EVENT | SIGNED BY | WHAT THE RECEIVING SYSTEM CHECKS | WITHOUT IT |
|---|---|---|---|
| **Sale at the register** | The register's device certificate, plus the cashier's session | Price matches the published record; stock is reduced once; payment is a token, never card data | Stock drifts, prices are disputed, and card data enters PCI scope |
| **Return at the counter** | The register and cashier, citing the original sale's signed record | The return matches a real sale and is within policy; an RMA number is issued | Refunds on items never sold there: return fraud |
| **RMA authorised** | A service agent, or an AI agent under a mandate | Refund limit, market and reason within the grant | Agents, contractors or plugins issuing refunds without authority |
| **Item received at the warehouse** | The scanner's device certificate | The item matches the RMA and its condition is graded | A refund paid before the item arrives, or stock counted twice |
| **Refund released** | The payment service, only after the receipt record exists | One refund per RMA, enforced by an idempotency key | Double refunds when a sync retries |
| **Store goes offline** | The register queues its signed events, each with an ISO 8601 UTC timestamp | Events replay in order; duplicates are dropped by key | Lost sales and returns, and a store that never reconciles with the warehouse |

The same rules from the rest of this article apply: a certificate per register and scanner, revocable one at a time; no plugin writing stock or refunds around the gate; and a ledger that shows, for any refund, which register, which person or agent, and which warehouse receipt it rested on.

## 5. Knowledge base: publish, don't unlock

Chatbots that answer from locked PDFs, docs, Markdown and spreadsheets, or from raw images and 3D files (JPG, PNG, SVG, WebP, OBJ, STL, FBX, GLB), read sources nobody approved, signed or versioned. A knowledge base for AI is **published once** as structured records carrying issuer, version, market, locale and consent scope, hashed and signed with a key you own. Media and 3D files become a manifest (alt text, dimensions, product link, licence) while the file stays in the vault under its own key.

The chatbot then climbs a decision ladder on that evidence: retrieval (RAG), corrective retrieval when evidence is weak (CRAG), a locale or personalisation adapter (LoRA) only where consent allows, and a hand-off to a person when the answer carries money, law or risk. No rung opens a locked file, no agent acts without a mandate, and price and stock come live from the system of record, never from embeddings. More in The AI ladder.




## Chatbot plugins: what they can't include

Shopify app-store chat plugins and enterprise suites such as Sprinklr run the bot as a third-party script with the vendor's own storage, keys and certificates. That makes them a signing-keys-by-proxy problem of their own: whatever the shopper types lands in the vendor's transcripts, in the vendor's region, under the vendor's CA. Three things should stay out of them.

| WHAT | TYPICAL CHAT PLUGIN OR SPRINKLR BOT | PUBLISHED-KB CHATBOT |
|---|---|---|
| **Card data (PCI)** | Must not be typed into chat: a transcript holding card numbers puts the vendor, and you, in PCI DSS scope, and masking depends on the vendor | Never accepted. Payment happens only in Shopify checkout |
| **Personal data (PII)** such as address, email, phone or ID numbers | Kept in the vendor's transcripts and region, outside your consent ledger, and missed when a deletion request runs | Not collected in chat. Identity comes from the signed-in account; consent is attached and deletion propagates |
| **Checkout links** | A generic product or cart link, without the shopper's market, consent state or grant | A URL hand-off to Shopify checkout with market and consent resolved; payment never in chat |
| Knowledge source | The vendor's copy of your help content, or private files uploaded to it | Published, signed records; Sprinklr content is migrated into them, then retired as a live source |
| Keys and certificates | The vendor's CA and keys, inherited by proxy | Keys you own, verifiable offline |

Check each vendor's terms and data-processing addendum for where transcripts are stored and how card numbers are masked; the point is that none of those choices are yours.

#### Chatbot build requirements

A headless Shopify proof of concept, built on public product feeds and chatbot articles, shows the gap this section describes. It uses public data on purpose, to show the difference between being visible and being governed. Anything public can be viewed by AI, but that does not incorporate it into a knowledge base corpus with rules and permissions: no issuer, version, locale or consent scope, and no grant saying who may use it for what. Public articles an AI can read are still ungoverned until they are published as records.

**Why an ungoverned ladder hallucinates.** A chatbot or AI escalation ladder that retrieves from public docs without a governed corpus pulls *anything* that matches: an old article, another market's price, a machine translation, a page written for a different audience. With no guardrails, no global ISO rules and no personalised consent state, it has nothing to check an answer against, so it fills the gaps:

| MISSING | WHAT THE LADDER DOES WITHOUT IT | GOVERNED CORPUS |
|---|---|---|
| **Guardrails** | Answers from any matching page, including retired or off-topic ones, and blends them | Retrieval reads published records only; weak evidence goes to CRAG or a person, never a guess |
| **Global ISO rules** | Mixes countries, languages, currencies and dates: a US price in a Korean answer, a stale date shown as current | Every record keyed by ISO 3166 country, BCP 47 language, ISO 4217 currency and ISO 8601 UTC time; retrieval filters on them |
| **Personalised consent state** | Personalises for everyone, or for no one, and can't tell which | Each answer checks the shopper's consent at the moment it's given; the locale adapter by default, personalisation only with consent |
| **Provenance** | Cites nothing, or cites a page that says something else | Each answer names its record, its issuer and its grant |

- ☐ **Migrate** each Sprinklr article into a canonical record (one answer per question), then stop reading Sprinklr live
- ☐ **Publish** every record with issuer, version, market, locale and consent scope, hashed and signed with a key you own
- ☐ **Re-index** retrieval from the published records only, filtered by locale
- ☐ **Hand off** checkout by URL to Shopify; take no card data and no personal data in chat
- ☐ **Trace** every answer to its record, its issuer and its grant
- ☐ **Retire** Sprinklr as a source once the published set covers its questions

**LoRA audience training, linked to BigQuery and organic lookup**

Two things are easy to confuse. **Audiences** decide *who* gets a personalised answer; **LoRA adapters** decide *how* the model writes for them: tone, vocabulary, format. BigQuery holds the audience, organic lookup supplies the evidence of how each market actually asks, and the adapter is trained only on de-identified pairs with a training basis. Personal data never goes into model weights, because an erasure request can't reach inside a trained model.

```
BigQuery (consent-filtered, keyed by ISO country, language, currency, UTC time)
  ├─ audiences ─────→ GA4 export + Shopify orders → segments, pLTV (BigQuery ML)
  │                   → cohort labels, held below a minimum cohort size
  ├─ organic lookup ─→ Search Console bulk export: queries by country, language, device (aggreg
  │                   → KB gaps: questions searched but not answered → publish records
  │                   → evaluation set: real phrasing per market
  └─ training pairs ─→ redacted, de-identified question-and-answer pairs + cohort label
                      with training consent → clean room → LoRA adapter per market × cohort
                      → registry: dataset_hash, record_count, consent basis, retire_by
Chatbot: pickAdapter(locale, cohort, consent) → base model + adapter; facts from published recc
```

| SOURCE IN BIGQUERY | USED FOR | NEVER USED FOR |
|---|---|---|
| GA4 export, with Consent Mode states | Cohort membership counts; which questions lead to conversion | Individual training rows |
| Shopify orders and pLTV scores | Cohort labels, under a pseudonymous ID | Weights that contain a person |
| Search Console bulk export (organic queries) | Knowledge base gaps; evaluation phrasing and vocabulary per market | Targeting a person: the data is aggregated by query |
| Redacted helpdesk question-and-answer pairs | Training pairs for tone and format | Facts: those stay in published records, so erasure still works |

## Rules that keep it governed:

- **Training consent is its own scope.** Marketing or personalisation consent is not consent to train; only records with a training basis enter the clean room.
- **Minimum cohort.** A cohort below the threshold gets no adapter and no label; it falls back to the locale adapter.
- **In-country training.** Korean data trains only on Korean-hosted systems; each market's rule from section 6 applies to training as well as storage.
- **Retire and retrain.** Every adapter carries `retire_by`; a withdrawal is honoured by retraining without the withdrawn records, not by editing weights.
- **Organic lookup feeds the corpus first.** A question people search for that the knowledge base can't answer becomes a published record before it becomes training data, so the adapter changes how the answer is written, never what it says.

Background: Google audiences and conversions with Vertex AI, and where LoRA fits and the clean room.

## 6. Globalization: rules management by market

Each market sets whether data must stay in country, on what basis it may leave, and which Cloudflare and Xano options keep it where it belongs.

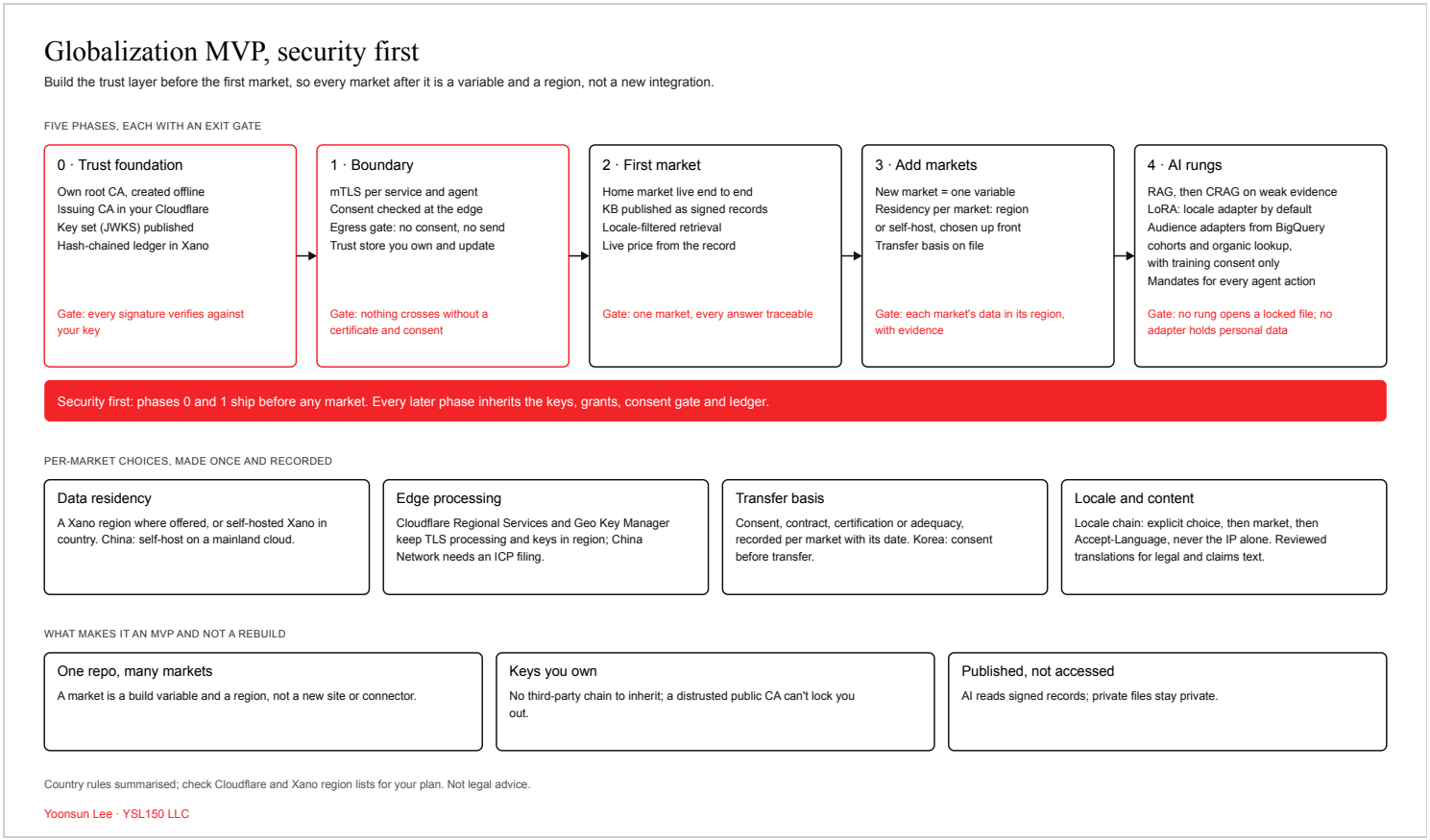
| MARKET | LAW | DATA IN COUNTRY | CROSS-BORDER TRANSFER | CLOUDFLARE | XANO |
|---|---|---|---|---|---|
| **China** | PIPL, Data Security Law, Cybersecurity Law | Required for critical infrastructure operators and above volume thresholds; important data stays | Security assessment, standard contract or certification, unless an exemption applies | China Network via JD Cloud (Enterprise), needs an ICP filing; Geo Key Manager picks key regions | No mainland region: self-host on a mainland cloud, with its own TLS certificate from a CA you choose |
| **Japan** | APPI | Not required by law | Consent naming the country, or a recipient with equivalent safeguards; EU adequacy | Regional Services and Geo Key Manager keep TLS processing and keys in Japan | A Japan-region instance where offered, or self-host in Japan |
| **Korea** | PIPA (2023 amendment) | No general rule; sector rules for finance and public-sector cloud | Consent before transfer, or a certified or adequate recipient; disclose country and recipient | Regional Services and Geo Key Manager; check the Korea region | A Seoul-region instance, or self-host in Korea |
| **Vietnam** | Personal Data Protection Law (from 2026), Decree 53 | Some services must store in Vietnam; foreign firms when ordered by the regulator | Transfer impact assessment filed with the Ministry of Public Security | No local residency product; keep personal data out of edge logs and caches | Self-host in Vietnam where localisation applies; otherwise a nearby region with the filing |
| **Singapore** | PDPA | Not required by law | Comparable protection: contract, binding corporate rules or certification | Regional Services and Geo Key Manager; check the region list | A Singapore-region instance where offered |

| MARKET | LAW | DATA IN COUNTRY | CROSS-BORDER TRANSFER | CLOUDFLARE | XANO |
|---|---|---|---|---|---|
| **Taiwan** | PDPA (2023 amendment) | Not general; sector rules | Allowed unless the regulator restricts a destination, e.g. mainland China for some sectors | Standard network; keep keys out of restricted regions | Nearest region or self-host; never route through a restricted destination |
| **EU / Germany** | GDPR, BDSG, TDDDG | Not required by law; often a contract term for public sector and health | Adequacy (including EU-US DPF, Japan, Korea, UK) or SCCs with a transfer assessment | Data Localization Suite: Regional Services, Customer Metadata Boundary, Geo Key Manager (EU) | An EU-region instance, or self-host in Germany |
| **UK** | UK GDPR, DPA 2018, Data (Use and Access) Act 2025 | Not required by law | UK adequacy regulations, IDTA or Addendum; UK-US data bridge | Regional Services; check the UK region | A UK or EU-region instance, or self-host in the UK |

Rules are summarised, not legal advice. Check Cloudflare's and Xano's current region lists for your plan; the AI cross-border requirements checklist goes deeper on transfer bases.

# 7. Globalization MVP, security first

Build the trust layer before the first market, so every market after it is a variable and a region, not a new integration.




| PHASE | BUILD | EXIT GATE |
|---|---|---|
| **0 · Trust foundation** | Own root CA created offline; issuing CA in your Cloudflare account; key set (JWKS) published; hash-chained ledger in Xano | Every signature verifies against your key |
| **1 · Boundary** | mTLS per service and agent; consent checked at the edge; egress gate (no consent, no send); a trust store you own and update | Nothing crosses without a certificate and consent |
| **2 · First market** | Home market live end to end; knowledge base published as signed records; locale-filtered retrieval; live price from the system of record | One market, every answer traceable |
| **3 · Add markets** | A new market is one build variable; residency per market (region or self-host) chosen up front; transfer basis on file | Each market's data in its region, with evidence |
| **4 · AI rungs** | RAG, then CRAG on weak evidence; LoRA with the locale adapter by default, and audience adapters trained from BigQuery cohorts and organic lookup only with training consent; mandates for every agent action | No rung opens a locked file; no adapter holds personal data |

Phases 0 and 1 ship before any market. Every later phase inherits the keys, grants, consent gate and ledger, which is what keeps it an MVP rather than a

rebuild per market.

### Publishing requirements for the MVP

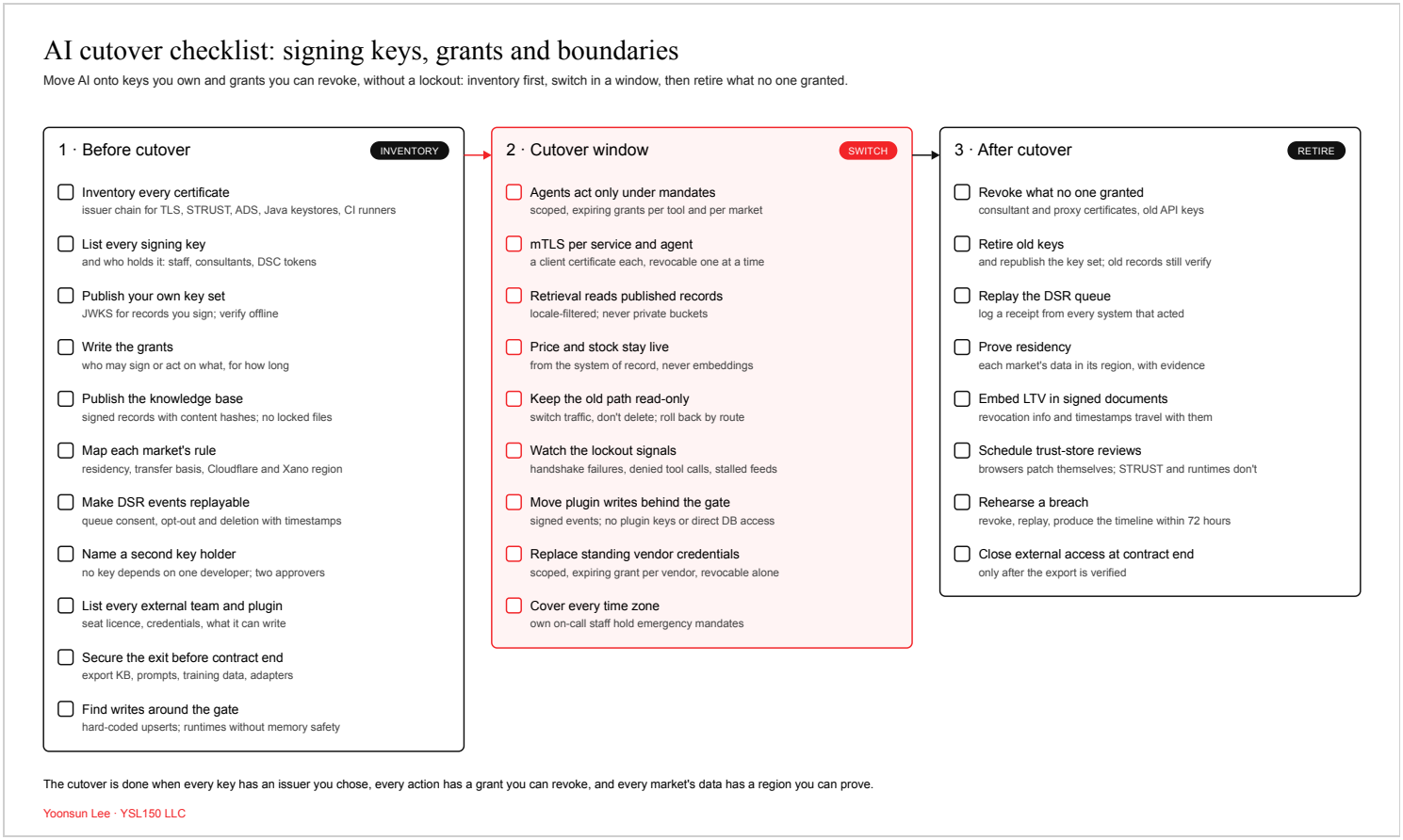
Four things are published in every market, and each is a signed record, not a file someone copies into a system:

| PUBLISHED | SOURCE AND FORMAT | SIGNED AND CHECKED | PHASE AND GATE |
|---|---|---|---|
| **Firmware** | The image plus its SBOM, per device class and version | Signed at build; vaulted under its own key; each download granted to a device certificate and recorded in the ledger; devices verify before install | Phase 1: no device runs an image it can't verify |
| **Markdown (MD)** | Markdown with frontmatter carrying issuer, version, market, locale and consent scope: the one source for the web page, the PDF, the book and the knowledge base records | Content-hashed and signed at publish; every surface traces back to the same commit | Phase 2: every answer and page traces to one source |
| **Permissions** | Grants and mandates as records: who or which agent, which action, which market, until when | Signed when issued, and again when revoked; checked on every event, never cached past expiry | Phase 1: nothing acts without a current grant |
| **Escalation ladders** | Per market: which sources RAG may read, the evidence bar that sends a question to CRAG, which LoRA adapters are allowed and under what consent, and which answers go to a person (money, law, risk) | Each ladder version reviewed (a QA attestation) and signed before it goes live; the version in force is recorded with every answer | Phase 4: no rung changes without a signed review |

A new market reuses all four: its firmware channel, its Markdown slice, its grants and its ladder are configuration on the same rails, not a new integration.

# 8. AI cutover checklist

Move AI onto keys you own and grants you can revoke, without a lockout: inventory first, switch in a window, then retire what no one granted.




### Before cutover: inventory

- ☐ **Inventory every certificate**, with its issuer chain: TLS, `STRUST`, ADS, Java keystores, CI runners
- ☐ **List every signing key** and who holds it: staff, consultants, DSC tokens
- ☐ **Publish your own key set** (JWKS) for the records you sign, so they verify offline
- ☐ **Write the grants**: who may sign or act on what, for how long
- ☐ **Publish the knowledge base** as signed records with content hashes; no locked files

- ☐ **Map each market's rule**: residency, transfer basis, Cloudflare and Xano region
- ☐ **Make DSR events replayable**: queue consent, opt-out and deletion with timestamps
- ☐ **Name a second key holder**: no key or rotation depends on one developer; two approvers and a written runbook
- ☐ **List every external team and plugin with access**: its seat licence, its credentials and what it can write
- ☐ **Secure the exit before the contract ends**: export the knowledge base, prompts, labelled data and adapters, and confirm the export opens
- ☐ **Find writes around the gate**: plugins with hard-coded upserts, or running without memory safety

#### Cutover window: switch

- ☐ **Agents act only under mandates**: scoped, expiring grants per tool and per market
- ☐ **mTLS per service and agent**: a client certificate each, revocable one at a time
- ☐ **Retrieval reads published records** only, locale-filtered; never private buckets
- ☐ **Price and stock stay live** from the system of record, never embeddings
- ☐ **Keep the old path read-only**: switch traffic, don't delete; roll back by route
- ☐ **Watch the lockout signals**: handshake failures, denied tool calls, stalled feeds
- ☐ **Move plugin writes behind the gate**: every write a signed event; no plugin holds signing keys or direct database access
- ☐ **Replace standing vendor credentials**: a scoped, expiring grant and certificate per vendor, revocable alone

☐ **Cover every time zone**: your own on-call staff hold pre-approved emergency mandates

#### After cutover: retire

☐ **Revoke what no one granted**: consultant and proxy certificates, old API keys

☐ **Retire old keys** and republish the key set; old records still verify

☐ **Replay the DSR queue** and log a receipt from every system that acted

☐ **Prove residency**: each market's data in its region, with evidence

☐ **Embed LTV** in signed documents, so revocation info and timestamps travel with them

☐ **Schedule trust-store reviews**: browsers patch themselves; `STRUST` and runtimes don't

☐ **Rehearse a breach**: revoke a key, replay the queue and produce the timeline within GDPR's 72 hours

☐ **Close external access at contract end**: only after the export is verified

#### Contract clauses to ask for

- **Disclose every provider**: every trust service provider and CA used for your signers or systems.
- **Certificates issued to the client**: certificates installed in your systems name you, not the consultancy.
- **Embed LTV**: revocation info and timestamps embedded, so documents stay verifiable after the relationship ends.

The cutover is done when every key has an issuer you chose, every action has a grant you can revoke, and every market's data has a region you can prove.

# Related

- Org Trust Layer: your own CA, fewer plugins
- The AI ladder: retrieval, adapters, humans, and who holds the keys
- Entitlement security
- Hydrogen or bespoke stack
- AI cross-border requirements checklist

*Patterns, not claims about any firm. eMudhra incident details per The Hacker News and Help Net Security, October 2026. Country rules summarised; not legal advice.*

---